E-COMMERCE CUSTOMER BEHAVIOR ANALYTICS

PROJECT STATUS

28 OCTOBER 2026

What is implemented so far

A web application for exploring retail transactions, segmenting customers, finding product associations, forecasting revenue, and now generating customer-level product recommendations. The original analytics are built around the UCI Online Retail dataset; a separate PostgreSQL setup and recommendation path have been added for Olist data.

541,909	8	2	2
UCI raw transactions	Core UCI tables	Supported data schemas for recommendations	Web application layers

Technology

Frontend	Backend	Database / analytics
React, TypeScript, Vite, Tailwind CSS, Recharts, Axios	Python, FastAPI, Pydantic, SQLAlchemy, Alembic	PostgreSQL, pandas, scikit-learn, mlxtend

Implemented capabilities

- **Data pipeline:** UCI Online Retail data import, cleaning, quality checks, cancellation handling, and persistence in PostgreSQL.
- **Customer and sales analysis:** dashboard summaries, revenue trends, top products, countries, and RFM customer scoring.
- **Machine learning and pattern mining:** K-Means customer groups, Apriori product associations, and daily revenue forecasts.
- **New recommendation feature:** a customer selector and personalized product suggestions for either the UCI or Olist schema.

Current data and scope

The core dashboard and batch analytics remain designed for UCI Online Retail transactions from 2010-2011. Olist, covering Brazilian marketplace transactions from 2016-2018, has a separate schema setup and is currently supported by the recommendation feature. The full analytics suite has not yet been migrated to Olist.

Core application: UCI Online Retail

The main application uses real transaction data for a UK non-store retailer, covering 1 December 2010 through 9 December 2011. The documented raw dataset has 541,909 rows and values in GBP. Import the data before running analyses; the app does not invent production dashboard values.

Area	What it does	Implementation
Import and cleaning	Loads the retail workbook or uploaded file; handles missing customer IDs, duplicates, cancellations, invalid quantities/prices, and product descriptions.	Pandas cleaning and database load; import run status is recorded.
Dashboard	Shows revenue, orders, average order value, customer and repeat-purchase counts, cancellations, trends, top products, and country summaries.	FastAPI endpoints query PostgreSQL; React charts render the results.
RFM segmentation	Scores customers by recency, frequency, and monetary value, then assigns named segments such as Champions and At Risk.	Quintile scoring with rank handling; results persist by analysis run.
Customer clustering	Groups customers with similar purchase patterns and reports cluster profiles.	K-Means on scaled log-transformed RFM data; silhouette scores guide k selection.
Market basket	Finds product combinations and rules such as items bought together.	Apriori and association rules, limited to frequent products to bound memory use.
Revenue forecasting	Forecasts daily revenue and compares model error with a seasonal-naive baseline.	Gradient Boosting with a chronological train/test split and future predictions.
Run history	Tracks imports and analytics from pending through completion or failure.	Background tasks store parameters, results, errors, and timestamps in PostgreSQL.

Database and API structure

The UCI schema has eight application tables: customers, products, orders, order_items, analytics_runs, customer_segments, association_rules, and forecasts. Foreign keys, indexes, a positive-price check, and a unique customer-per-segment-run constraint support the main query patterns. Alembic manages the schema; the FastAPI API exposes dashboard, customer, analysis, import, and run-history endpoints.

Analysis lifecycle

A user starts an analysis from the interface. The API records a pending run, a background task computes and stores results, and the interface polls the run until it completes or fails.

Olist database and recommendations

Olist data was selected as a richer public alternative. Its 2016-2018 Brazilian marketplace dataset contains related customer, seller, product, order, payment, review, location, and category-translation files. It provides delivery and review information that the original UCI dataset does not.

Olist setup added to the project

- **Separate database scripts:** optional role/database creation and PostgreSQL table/index creation in the *Databas_Olist* folder.
- **Relational schema:** nine Olist tables for customers, sellers, products, category translation, orders, order items, payments, reviews, and geolocation.
- **Import guide:** pgAdmin CSV import instructions, foreign-key-safe import order, generated geolocation ID handling, and row-count queries.
- **Local load:** the user reports importing the CSV files in pgAdmin; the process list showed completed jobs for all nine files.

Personalized recommendation implementation

- A new Recommendations page lets the user search/select a customer and view prior purchases and recommendations.
- Backend endpoints list eligible customers and return recommendations for a selected customer ID.
- The service detects the available schema: the original UCI tables or the Olist tables. For Olist, it groups order-specific IDs by customer_unique_id and uses delivered orders.
- Customers are compared using Jaccard similarity over distinct purchased products. Products owned by the selected customer are excluded; recommendations are ranked by similarity-weighted purchases from up to 100 similar customers.
- When similar-customer candidates are unavailable, the service falls back to popular delivered products that customer has not bought.

Olist product descriptions are not present in the public product file. The page therefore shows product IDs with English category labels when available. UCI results show the product descriptions.

Current integration gap and next work

Status	Details
Available now	Olist database setup scripts and guide; Olist-specific recommendation queries and customer/product recommendation UI; recommendations also work against the legacy UCI schema.
Still UCI-specific	Dashboard, RFM, clustering, Apriori, forecasting, original importer, and run-history schema still use tables such as *orders*, *customers*, and *order_items*. Olist has prefixed tables such as *olist_orders*. Pointing the whole app at Olist therefore causes the legacy dashboard endpoints to return missing-table errors.
Recommended next	First use the Recommendations page against the imported Olist database. To make Olist the full project dataset, implement an Olist importer and adapt the dashboard, analytics, and frontend to Olist fields and BRL currency; then validate with a full backend and frontend run.

Verification note

This report summarizes the code and setup work present in the project folder as of 28 October 2026. Automated tests and a frontend build were not run for this report. The running Olist connection still requires the correct PostgreSQL login in backend/.env; the database name and login role are separate URL fields.

Project reference files: README.md; backend/app/api.py; backend/app/services/recommendations.py; Databas_Olist/README.md; Databas_Olist/01_create_database.sql; Databas_Olist/02_create_tables.sql.